



Mata Kuliah : Pemrograman Web Lanjut (PWL)
Program Studi : D4 – Teknik Informatika
Semester : 4

Kelas : 2A
NIM : 244107020030
Nama : Athaulla Hafizh
Jobsheet Ke- : 10

Laporan Jobsheet 10 - 01

Link Repository : <https://github.com/athfizh/Pemrograman-Web-Lanjut>

Praktikum

Langkah	Jawaban/Deskripsi
1	Menambahkan Delete Action <pre>use Filament\Tables\Actions\BulkActionGroup; use Filament\Tables\Actions\CreateAction; use Filament\Tables\Actions\DeleteBulkAction; use Filament\Tables\Actions\DeleteAction; use Filament\Tables\Actions\EditAction;</pre>
2	Menambahkan Replicate (Copy) Action <pre>use Filament\Tables\Actions\ReplicateAction; ReplicateAction::make(),</pre>
3	Membuat Custom Action (Ubah Status Publish) 1. Tambahkan Custom Action



```
->recordActions([
  Action::make('status')
  ->label('Status Change')
  ->form([
    Checkbox::make('published')
    | ->default(fn ($record) => $record->published),
  ])
  ->action(function ($record, $data) {
    $record->update(['published' => $data['published']])
  }),
  ReplicateAction::make(),
  EditAction::make(),
  DeleteAction::make(),
]);
```

2. Tambahkan Form Input pada Action

```
Checkbox::make('published')
| ->default(fn($record): bool => $record->published),
```

3. Tambahkan Logic untuk Update Data

```
->recordActions([
  ReplicateAction::make(),
  EditAction::make(),
  DeleteAction::make(),
  Action::make('status')
  ->label('status change')
  ->schema([
    Checkbox::make('published')
    | ->default(fn($record): bool => $record->published)
  ])
  ->action(function ($record, $data) {
    $record->update(['published' => $data['published']])
  }),
]);

->toolbarActions([
  CreateAction::make(),
  BulkActionGroup::make([
    | DeleteBulkAction::make(),
  ]),
]);
```

4. Tambahkan Icon



```
->label('status change')  
->icon('heroicon-o-check-circle')  
->schema([
```

Langkah Praktikum

Langkah	Jawaban/Deskripsi
1	Tambahkan Delete & Replicate action <pre>->recordActions([ReplicateAction::make(), EditAction::make(), DeleteAction::make(), Action::make('status') ->label('status change') ->icon('heroicon-o-check-circle') ->requiresConfirmation() ->schema([Checkbox::make('published') ->default(fn(\$record): bool => \$record->published),]) ->action(function (\$record, \$data) { \$record->update(['published' => \$data['published']]); })),)</pre>
2	Buat custom action untuk toggle publish/unpublish <pre>Action::make('status') ->label('status change') ->icon('heroicon-o-check-circle') ->requiresConfirmation() ->schema([Checkbox::make('published') ->default(fn(\$record): bool => \$record->published),]) ->action(function (\$record, \$data) { \$record->update(['published' => \$data['published']]); })),</pre>
3	Tambahkan icon berbeda untuk tiap action



	<pre>->label('status change') ->icon('heroicon-o-check-circle') ->requiresConfirmation()</pre>
4	Tambahkan confirmation pada custom action <pre>Action::make('status') ->label('status change') ->icon('heroicon-o-check-circle') ->requiresConfirmation()</pre>

Analisis dan Diskusi

1. Mengapa action di tabel lebih efisien dibanding halaman edit?

Action di tabel memungkinkan pengelolaan data yang cepat dan presisi tanpa perlu memuat ulang keseluruhan halaman (pindah ke URL edit) atau merender semua field input dari resource tersebut. Hal ini sangat efisien untuk aksi-aksi yang sifatnya tunggal seperti menghapus, menduplikasi, memicu sebuah job, atau mengubah status tertentu (toggle publish). Admin menghemat banyak klik dan waktu, sehingga produktivitas meningkat.

2. Apa perbedaan predefined action dan custom action?

- Predefined Action: Adalah action bawaan (built-in) yang sudah disediakan oleh Filament (seperti `EditAction::make()`, `DeleteAction::make()`, `ReplicateAction::make()`). Semua logika database, notifikasi keberhasilan, ikon, hingga konfigurasi konfirmasi telah ditangani oleh Filament secara default, tanpa perlu kita definisikan ulang isinya.

- Custom Action: Adalah action fleksibel yang dibuat menggunakan `Action::make('nama_action')`. Pada custom action, kita harus mendefinisikan label, ikon, form (schema) jika diperlukan, dan logika penanganan data (`->action(function() {})`) sepenuhnya dari nol (dari awal) sesuai business logic spesifik yang kita inginkan.

3. Bagaimana cara menambahkan validasi dalam custom action?

Validasi pada custom action bisa ditambahkan langsung di dalam komponen field yang dirender melalui `->schema([...])`. Anda dapat menautkan metode-metode validasi bawaan Filament seperti di form builder biasa.

4. Kapan kita menggunakan Replicate?



ReplicateAction sangat ideal digunakan pada skenario di mana pengguna (admin) sering kali harus membuat suatu data baru yang sebagian besar isi informasinya sama persis dengan record data yang sudah ada sebelumnya. Misalnya:

- Menginput ratusan produk yang spek-nya mirip namun berbeda warna/varian.
- Menyalin template artikel atau dokumen yang formatnya berulang. Daripada mengetik atau memilih opsi yang sama dari awal untuk form yang sangat panjang, admin cukup menduplikasi (replicate) record yang ada lalu melakukan modifikasi kecil yang diperlukan sebelum disimpan.